GrowthOS AI Module (AI Growth Brain): Definitive Architecture & Technical Specification

Strictly NO emojis in code, docstrings, terminal output, or logs.

---

1. Executive Vision: What the AI Module Is, Does, and Is Meant to Do

The **AI Growth Brain (Module 4)** is the central autonomous decision, semantic reasoning, predictive analytics, and policy orchestration engine of GrowthOS.

1.1 What It Does

- **Ingests and Authenticates Telemetry**: Ingests high-frequency micro-behaviors, clicks, cart modifications, and conversions from the storefront SDK using ephemeral cryptographic tokens.
- **Calculates Deterministic Ground-Truth Facts**: Computes conversion funnels, revenue metrics, Average Order Value (AOV), and 7-day comparative performance windows in pure Python/SQL without LLM math.
- **Detects Anomalies and Friction**: Identifies conversion drop-offs, rage clicks, payment gateway outages, and campaign bleeders using statistically validated tests.
- **Synthesizes 3-Tier Answers for Merchants**: Answers complex natural language business questions using a 3-tier response framework (Tier 1: Ground-Truth Fact, Tier 2: Semantic Analysis, Tier 3: Recommended Action), strictly verified by an anti-hallucination evidence linker.
- **Orchestrates Interventions Across Modules**: Automatically triggers retention fee waivers in Module 1, dense telemetry sampling in Module 2, ad budget rebalancing in Module 3, and recovery messages.
- **Enforces Merchant Policies and Governance**: Evaluates all proposed actions against daily spend caps, customer contact cooldowns, DPDP consent records, and quiet hours before dispatching.
- **Evaluates Causal Attribution**: Measures true incremental lift using pre-policy Intent-To-Treat (ITT) holdout control groups evaluated over a standardized 24-hour attribution window.

1.2 What It Is Meant to Do (The Autonomous Closed-Loop Vision)

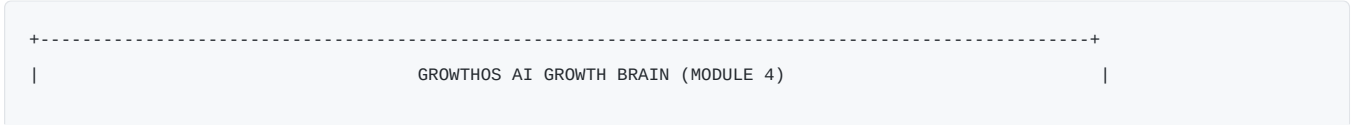
In traditional e-commerce architectures, customer acquisition (Ads), conversion (Storefront), and retention (Loyalty) operate in disconnected silos. GrowthOS unites these domains into a self-optimizing closed loop:

1. **Acquisition (Module 3)** drives traffic to the storefront.
2. **Behavior (Module 2)** detects when a visitor hesitates at checkout due to high delivery fees or friction.
3. **AI Brain (Module 4)** identifies the hesitation, checks customer loyalty tier and consent, and proposes a personalized delivery fee waiver or bonus point retention action.
4. **Retention (Module 1)** executes the waiver, converting the abandoned cart into a completed order.
5. **Attribution (Module 4 Closing Worker)** credits the purchase back to the acquisition campaign and loyalty policy, rebalancing ad spend away from underperforming campaigns toward high-converting cohorts.

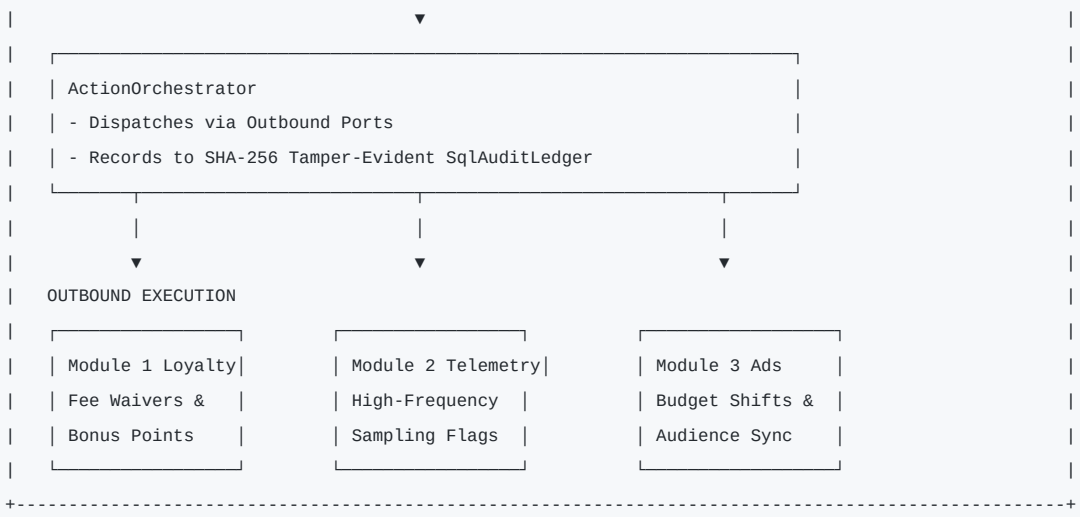
---

2. System Architecture & High-Level Topology

The AI Module follows a clean **Hexagonal Architecture (Ports and Adapters)** to decouple core business logic and decision engines from database technologies, LLM providers, and external microservice APIs.




INBOUND FEEDS
Module 2 Behavior Telemetry
(Ephemeral Token Auth)
Verified Orders & Refunds
(Ad-block resilient feed)
Telemetry Ingestion & Event Storage
- Ephemeral HMAC session token verification
- Deduplication & Idempotency key tracking
- SqlEventStorage (Append-only event envelope repository)
ANALYTICAL & DECISION CORE
MetricsEngine
- 7d Current vs 7d Prior
- Pure deterministic math
DigestEngine
- Z-test anomaly detection
- Friction spike detection
NLQueryEngine
- Intent classification
- Fact retrieval templates
Decision Providers
- Gemini Live Provider
- OpenRouter Fallback
- Deterministic Rule Engine
EvidenceLinker
- Exact count matching (0%)
- Display rate tolerances
- Anti-hallucination filter
GOVERNANCE & ORCHESTRATION
HoldoutEngine (Pre-Policy ITT Split)
- Randomly assigns 10% to control arm before treatment or policy
PolicyEngine & Governance Gates
- DPDP Consent verification (ConsentStore source of truth)
- Daily budget spend caps (Integer paise)
- Customer contact cooldowns & Quiet hours (21:00-09:00 IST)



---

3. Cross-Module Integrations: How Each Module Interacts with the AI Brain

3.1 Module 1: Loyalty & Rewards Platform Integration

- **Outbound Port Contract**: `LoyaltyPort` (`ports/outbound.py`)
  - `waive_delivery_fee(customer_id, fee_amount, points_deducted, idempotency_key)`
  - `grant_bonus_points(customer_id, points, reason, idempotency_key)`
  - `issue_discount_coupon(customer_id, coupon_code, discount_amount, idempotency_key)`
- **Adapter Implementation**: `HttpLoyaltyAdapter` (`adapters/loyalty_waivers_adapter.py`)
  - Authenticates via HMAC-SHA256 request signatures.
  - Propagates client-generated idempotency keys (`Idempotency-Key` HTTP header).
  - Maintains in-memory circuit breaker tripping after 3 consecutive failures.
- **Business Rationale & Logic**:
  - When the AI Brain detects high friction at checkout caused by shipping fees (e.g. cart INR 450, delivery fee INR 50), it checks customer loyalty eligibility.
  - If the customer has sufficient points (e.g. 40 points) or tier privileges (Gold/Platinum), the Brain dispatches a fee waiver.
  - The waiver is checked by `PolicyEngine` against the merchant's daily fee waiver budget cap (e.g. INR 2,500/day) and a 48-hour customer cooldown window.

3.2 Module 2: Storefront & Behavior SDK Integration

- **Inbound Telemetry**:
  - Browser-side SDK captures micro-behaviors: scroll velocity, element dwell times, rage clicks, exit intent mouse trajectories, and form field abandons.
  - Events are transmitted as structured `EventEnvelope` payloads.
- **Ephemeral Session Token Authentication**:
  - Browser cannot securely hold persistent API secrets.
  - Telemetry endpoints validate short-lived, server-minted HMAC session tokens (`SessionTokenVerifier`).
  - Tokens bind `session_id` , `tenant_id` , and expiry timestamp, preventing unauthorized event injection while permitting client-side streaming.
- **Outbound Port Contract**: `BehaviorPort` (`ports/outbound.py`)
  - `request_dense_telemetry(component_selector, duration_minutes, sample_rate)`
  - When an anomaly is detected on a specific checkout button, the Brain commands the SDK to elevate telemetry sampling rate on that DOM node.

3.3 Module 3: Ads Management & Optimization Integration

- **Outbound Port Contract**: `AdsPort` (`ports/outbound.py`)
  - `shift_campaign_budget(campaign_id, delta_amount, current_budget, idempotency_key)`
  - `pause_underperforming_campaign(campaign_id, reason, idempotency_key)`
  - `sync_audience_segment(segment_name, customer_ids, action)`
- **Cross-Module Budget Governance**:
  - In earlier prototypes, Module 3 and Module 4 both independently shifted ad budgets, creating budget oscillation and double-spending.
  - In the current architecture, Module 3 owns campaign-level CPC/keyword management, while Module 4 acts as the strategic governance authority.
  - All budget shifts are strictly limited to $\leq \pm 15\%$ per 24-hour cycle, subject to a minimum 3-day baseline evaluation and 1-click snapshot rollback.

3.4 External Feeds: Verified Multi-Channel Orders & DPDP Consent Store

- **Verified Order Feed (`MockOrderPort` , `SyntheticOrderIngester`)**:
  - Telemetry events from browsers fail when visitors use ad-blockers, clear cookies, or complete purchases via third-party payment redirect webhooks.
  - The AI Brain ingests a separate, server-verified order feed directly from order management systems and payment gateways.
  - Compensates for ad-blocked revenue, handles refunds, and reconciles cancellations.
- **Consent Source of Truth (`ConsentStore` , `SqlConsentAdapter`)**:
  - Enforces India's Digital Personal Data Protection (DPDP) Act 2023.
  - Maintains persistent records of customer consent granted by purpose (`analytics` , `marketing` , `retention` , `telemetry`).
  - Proactive retention messages and personalized fee waivers fail-closed if consent for `retention` is absent or withdrawn.

---

4. Deep Engine Breakdown: Logic and Architectural Rationales

4.1 Telemetry Ingestion & Ephemeral Token Verification (`services/ingestion_service.py`)

- **WHAT**: Ingests, validates, deduplicates, and persists storefront event envelopes.
- **HOW**:
  - Validates `tenant_id` , `event_type` , `event_id` , and timestamps.
  - Verifies token signature: `HMAC-SHA256("session_id:tenant_id:expires_at", server_secret)` .
  - Rejects expired tokens ($t > expires_at$) and mismatches fail-closed.
  - Deduplicates events using in-memory or database unique keys.
- **WHY**: Raw telemetry cannot be trusted implicitly. Browser secrets can be extracted by users. Ephemeral session tokens guarantee that event streams originate from genuine server-minted visitor sessions without exposing merchant master credentials.

4.2 Deterministic Metrics Engine (`engines/metrics_engine.py`)

- **WHAT**: Calculates business metrics, conversion funnels, and comparative windows.
- **HOW**:
  - Computes two aligned 7-day windows: Current ($T-7 \dots T$) vs Prior ($T-14 \dots T-7$).
  - Aligns day boundaries to Indian Standard Time (IST, UTC+05:30) midnight.
  - Calculates funnel transitions:

$$\text{Conversion} = \frac{\text{Numerator}}{\text{Denominator}}$$

  - Zero generative math: all calculations are executed in Python integers and floats; LLMs receive read-only numbers.
- **WHY**:
  - LLMs hallucinate math, especially percentages, rounding, and ratios.
  - Fixed 7-day windows account for day-of-week seasonality (e.g. Sunday surges vs Tuesday dips).
  - IST alignment ensures midnight cutoffs correspond to real merchant operating days rather than UTC mid-day splits.

4.3 Digest & Anomaly Engine (`engines/digest_engine.py`)

- **WHAT**: Detects statistically significant drops, spikes, and performance anomalies.
- **HOW**:
  - Evaluates metric deltas between current and prior windows.
  - Uses two-proportion Z-tests for conversion rates:

$$Z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}$$

  - Flags anomalies where $|Z| \geq 2.576$ ($p < 0.01$) or percentage drop exceeds vertical thresholds (e.g. -10%).
- **WHY**:
  - Simple percentage change comparisons produce excessive false alarms on low-volume metrics.
  - Proportion-based Z-testing prevents flagging random noise as business emergencies.

4.4 Anti-Hallucination Evidence Linker (`engines/evidence_linker.py`)

- **WHAT**: Verifies that every claim in an LLM-generated narrative is backed by deterministic facts.
- **HOW**:
  - Parses numerical claims (counts, percentages, monetary amounts) from generated text using regex.
  - Cross-references each claim against ground-truth facts extracted from `MetricsEngine`.
  - **Count Policy**: Counts must match with **0.0% tolerance** (exact integer match). An LLM stating "142 orders" when ground truth is 141 is flagged.
  - **Rate Policy**: Percentages and rates must match at the displayed precision (e.g. 2.4% matches 2.42%, but not 2.6%).
  - Sentences containing ungrounded or contradictory numbers are pruned or rejected.
- **WHY**:
  - Merchant trust is completely destroyed if an executive digest cites inaccurate order counts or phantom revenue.
  - Strict zero-tolerance count validation guarantees that narrative digests remain legally and operationally accurate.

4.5 Heuristic Scoring & Friction Classifier (`engines/heuristic_scoring.py`)

- **WHAT**: Computes real-time friction scores and customer propensity scores.
- **HOW**:
  - Evaluates session signals:

$$\text{FrictionScore} = w_{\text{rage}} \cdot N_{\text{rage}} + w_{\text{hesitation}} \cdot T_{\text{dwell}} + w_{\text{err}} \cdot N_{\text{errors}}$$

  - Classifies delivery fee sensitivity based on cart value relative to free shipping threshold and customer past behavior.
- **WHY**:
  - Fast, deterministic heuristics evaluate within $< 5\text{ms}$, allowing immediate modal intervention before the visitor exits the page, without waiting for high-latency LLM round-trips.

4.6 Natural Language Query Engine & 3-Tier Answers (`engines/nl_query_engine.py`)

- **WHAT**: Answers merchant analytical questions with grounded facts.
- **HOW**:
  - Classifies user question into parameter templates (e.g. `metric_comparison`, `funnel_breakdown`, `roi_audit`).
  - Retrieves grounded facts from `MetricsEngine`.
  - Prompts LLM to structure answers into three mandatory tiers:
    1. **Tier 1: Ground-Truth Fact**: The verified answer with citations (e.g. "Conversion dropped by 3.2% from 14.1% to 10.9%").
    2. **Tier 2: Semantic Analysis**: Root-cause context (e.g. "Drop is concentrated in Mobile Web checkout on payment gateway").
    3. **Tier 3: Recommended Action**: Specific, policy-governed intervention (e.g. "Deploy delivery fee waiver for carts above INR 400").
  - Passes output through `EvidenceLinker` before returning to merchant.
- **WHY**:
  - Prevents prompt injection by banning raw text-to-SQL generation.
  - Gives merchants immediate direct answers first, followed by actionable strategic recommendations.

4.7 Decision Providers (`adapters/gemini_decision_provider.py` , `openrouter_decision_provider.py` , `rule_decision_provider.py`)

- **WHAT**: Multi-tier reasoning providers for strategic decision formulation.
- **HOW**:
  - **Primary**: Google Gemini Live Provider with structured JSON function calling schemas.
  - **Secondary**: OpenRouter Provider for multi-model failover (Claude, Llama, DeepSeek).
  - **Tertiary (Degraded Mode)**: `RuleDecisionProvider` executing purely deterministic rule trees when external APIs are offline or rate-limited.
- **WHY**:
  - System resilience: Outages in external LLM providers never take down GrowthOS store operations; decisions automatically fall back to deterministic rule providers.

4.8 Holdout Engine & Causal ITT Assignment (`engines/holdout_engine.py`)

- **WHAT**: Assigns customers to treatment or control arms for causal lift measurement.
- **HOW**:
  - Assigns customers to control arm (10%) using deterministic SHA-256 hashing:

$$\text{Hash}(\text{customer_id} + \text{salt}) \pmod{100} < 10 \implies \text{Control}$$

  - **Crucial Rule**: Holdout is assigned **pre-policy** and before any approval or autonomy gates.
  - Control customers receive the status quo (no fee waiver, no discount) regardless of whether the system operates in Manual, Approval, or Autonomous mode.
- **WHY**:
  - If holdouts are assigned only in Autonomous mode or after policy filters, the control group suffers from severe selection bias. Pre-policy Intent-To-Treat (ITT) guarantees an unbiased baseline for true incremental lift measurement.

4.9 Policy Engine & Merchant Governance (`engines/policy_engine.py`)

- **WHAT**: Deterministic gatekeeper verifying business rules before any action dispatches.
- **HOW**:
  - Validates:
    1. **Budget Caps**: Integer paise maximums for discounts and fee waivers.
    2. **Cooldown Windows**: Minimum hours (e.g. 48h) between interventions for the same customer.
    3. **Quiet Hours**: Banned outbound communications between 21:00 and 09:00 IST.
    4. **DPDP Purpose Consent**: Fail-closed verification against `ConsentStore` .
- **WHY**:
  - Safety-first autonomous operation: The AI Brain is forbidden from executing arbitrary actions. Every decision must pass strict merchant-defined constraints.

4.10 Action Orchestrator & Cryptographic Audit Ledger (`services/action_orchestrator.py` , `adapters/sql_audit_ledger.py`)

- **WHAT**: Executes approved actions and records tamper-evident audit logs.
- **HOW**:
  - Dispatches actions via outbound ports with cryptographic HMAC signatures.
  - Appends every action to `SqlAuditLedger` .
  - Links audit records into a cryptographic SHA-256 hash chain:

$$\text{Hash}_n = \text{SHA256}(\text{Hash}_{n-1} + \text{RecordData}_n)$$

- **WHY**:
  - Non-repudiation: Merchants and auditors can verify that audit logs have never been modified, deleted, or forged.

4.11 24-Hour Attribution Closing Worker (`services/attribution_closing_worker.py`)

- **WHAT**: Reconciles and attributes downstream conversions to past interventions.

- **HOW**:
  - Evaluates conversions within a unified 24-hour attribution window.
  - Compares conversion rates between Treated (T) and Control (C) arms:

$$\text{Lift} = \frac{p_T - p_C}{p_C}$$

  - Computes two-sample Z-statistics and 95% confidence intervals.
- **WHY**:
  - Eliminates attribution window conflict (earlier 30-day ad windows vs 24-hour retention windows).
  - Proves whether interventions actually generated incremental profit or merely subsidized organic purchases.

4.12 Idle Cart Sweep Worker (`services/idle_cart_sweep_worker.py`)

- **WHAT**: Background daemon identifying stalled sessions and triggering recovery.
- **HOW**:
  - Sweeps active sessions where cart idle time exceeds threshold (e.g. 15 minutes).
  - Evaluates customer loyalty eligibility, consent, and quiet hours.
  - Queues recovery notifications or loyalty fee waivers.
- **WHY**:
  - Prevents revenue leakage by reviving high-intent abandoned shopping carts automatically.

4.13 Provider Benchmark Evaluator (`evaluation/benchmark_evaluator.py`)

- **WHAT**: Evaluates decision providers against non-circular benchmark suites.
- **HOW**:
  - Compares model choices against independent ground-truth outcomes rather than self-referential rules.
  - Computes **Counterfactual Regret**:

$$\text{Regret} = \text{Utility}(\text{OptimalAction}) - \text{Utility}(\text{ChosenAction})$$

- **WHY**:
  - Prevents trivial 100% scores where models are evaluated against the very rules that created the test cases.

4.14 Randomized Scenario Test Environment (`sim/`)

- **WHAT**: Seeded simulation framework for testing all four modules.
- **HOW**:
  - Uses NumPy `SeedSequence` spawning independent random streams for tenants, loyalty, traffic, ads, and faults.
  - Supports `--freeze <domain>` to isolate single-component behaviors.
  - Runs simulations against independent exact oracles (`ExactMetricsOracle` , `ExactLoyaltyOracle`) and fail-closed invariant checkers (`InvariantChecker`).
- **WHY**:
  - Enables 100% deterministic replay of complex edge-case failures and guarantees system correctness across thousands of merchant permutations.

---

5. Security, Cryptography & Compliance

Layer	Mechanism	Implementation	Guarantee
Telemetry Auth	Ephemeral HMAC Tokens	`SessionTokenVerifier` (`services/ingestion_service.py`)	Prevents unauthenticated browser event spoofing without exposing secrets
Audit Ledger	SHA-256 Hash Chain	`SqlAuditLedger` (`adapters/sql_audit_ledger.py`)	Tamper-evident ledger; modification breaks cryptographic chain

Layer	Mechanism	Implementation	Guarantee
Outbound Signing	HMAC-SHA256 Signatures	`HttpLoyaltyAdapter`, `HttpAdsAdapter`	Target microservices authenticate requests and reject replays
Idempotency	UUID Idempotency Keys	Outbound Port headers (`Idempotency-Key`)	Network retries cause zero duplicate state mutations
Data Privacy	DPDP Act 2023 Consent	`ConsentStore` (`adapters/sql_consent_adapter.py`)	Actions fail-closed if purpose consent is missing or withdrawn

---

6. Mathematical Invariants & Verification Guarantees

1. **Integer Money Invariant**:

$$\forall \text{amount} \in \{\text{cart}, \text{fee}, \text{discount}, \text{budget}\} : \text{amount} \in \mathbb{Z}_{\geq 0} \text{ (paise)}$$

2. **Loyalty Conservation Invariant**:

$$\forall c \in \text{Customers} : \text{ClosingPoints}_c = \text{OpeningPoints}_c + \sum \text{Earned}_c - \sum \text{Redeemed}_c$$

3. **Loyalty Non-Negativity Invariant**:

$$\forall c \in \text{Customers} : \text{ClosingPoints}_c \geq 0$$

4. **Waiver Cooldown Invariant**:

$$\forall c, \forall i < \text{len}(\text{waivers}) - 1 : t_{i+1} - t_i \geq \text{CooldownHours} \times 3600$$

5. **Funnel Monotonicity Invariant**:

$$N_{\text{view}} \geq N_{\text{cart}} \geq N_{\text{checkout}} \geq N_{\text{payment}} \geq N_{\text{order}}$$

6. **Anti-Hallucination Count Invariant**:

$$|\text{ReportedCount} - \text{GroundTruthCount}| = 0$$

7. **Budget Rebalance Invariant**:

$$|\Delta \text{Budget}_{\text{daily}}| \leq 0.15 \times \text{CurrentBudget}_{\text{daily}}$$